PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRODUCT DEVELOPMENT AND CLIENT SUPPORT USING JAVA AND SPRING BOOT

AN INTERNSHIP REPORT

Submitted by

RUKMINI SURVE M - 20221COM0187

Under the guidance of,

Mr. IRFAN RAJAB BHAT

BACHELOR OF TECHNOLOGY

IN

COMPUTER ENGINEERING

GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report **"Product Development and Client support using Java and Spring Boot"** is a Bonafide work of **RUKMINI SURVE M (20221COM0187),** who has successfully carried out the internship work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER ENGINEERING during 2025-2026.

Mr. Irfan Rajab Bhat
Internship Guide
PSCS
Presidency University

Mr. Sreehari T M
Program Internship Coordinator
PSCS
Presidency University

Dr. Md Ziaur Rahaman
School Internship Coordinator
PSCS
Presidency University

Dr. Gopal Krishna Shyam
Head of the Department,
PSCS
Presidency University

Dr. Duraipandian N
Dean
PSCS / PSIS
Presidency University

Name and Signature of the Examiners

Sl. No	Name	Signature	Date
1			
2			

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I am a student of final year B.Tech in COMPUTER ENGINEERING , at Presidency University, Bengaluru, named RUKMINI SURVE M, hereby declare that the internship work titled **"Product Development and Client Support using Java and Spring Boot"** has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER ENGINEERING during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

RUKMINI SURVE M USN: 20221COM0187 Signature

PLACE: BENGALURU

DATE: 24 - April 2026

INTERNSHIP COMPLETION CERTIFICATE

Encrypt Infotech

Certificate of Internship
TO WHOM IT MAY CONCERN.

This is to certify that **Ms. Rukmini Surve M.** has successfully completed the internship program as an **Associate Software Engineer** from 05.01.2026 to 12.04.2026. She worked on Product Development and Client Support during her internship.

She took keen interest in the work assigned and successfully completed it. During the period of internship, we found her to be punctual, hardworking and inquisitive.

We wish her luck and success in all her future endeavors.

Sincerely,

For Encrypt Infotech LLP.
SHYLAJA GOVINDARAJ
Managing Director

For ENCRYPT INFOTECH LLP
Shylaja
Authorised Signatory

ENCRYPT INFOTECH LLP
38/1, 38/2, 4th floor, Vinayaka high point, 1st main, 4th phase Yelahanka new town,
Bangalore- 560064 PH :9620720700,9916858790

ACKNOWLEDGEMENTS

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the internship.

I would like to sincerely thank my internal guide **Mr. Irfan Rajab Bhat, Assistant Professor**, Presidency School of Computer Science and Engineering, Presidency University, for his moral support, motivation, timely guidance and encouragement provided to me during the period of internship work.

I am also thankful to **Dr. Gopal Krishna Shyam, Professor, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for his mentorship and encouragement.

I express my cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS, **Dr. Shakkeera L**, Associate Dean, Presidency School of Computer Science and Engineering and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

I am grateful to **Dr. Md Ziaur Rahaman,** Internship Coordinator and **Mr. Sreehari T M,** Program Internship Coordinator, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

Rukmini Surve M

ABSTRACT

The rapid growth of enterprise software solutions has led to increasing demand for robust, scalable, and efficient web applications capable of supporting real-time business operations. Organizations across industries such as construction, manufacturing, and logistics rely heavily on centralized digital platforms to manage supply chains, client interactions, financial transactions, and operational data.

This internship was conducted at Encrypt Infotech LLP, a product-based IT company headquartered in Bengaluru, India, established in 2017 and specializing in the development and maintenance of enterprise-grade software solutions.

The internship, titled "Product Development and Client Support using Java & Spring Boot," focused on contributing to two primary enterprise products: Any Crusher and ERMC (Enterprise RMC Management System). The ERMC platform, hosted at e-rmcpro.com, is a comprehensive web-based solution designed for Ready-Mix Concrete (RMC) plant management. It encompasses a wide range of functional modules including Invoice Management, Delivery Challan, Customer Relationship Management (CRM), Supplier Master, Inventory Store, Fleet Store, Cash and Finance, Petty Cash, HR Management, Grade and Product configuration, Weighment integration, and a real-time operational Dashboard.

The core technical work involved understanding and navigating large existing Java codebases, resolving real-time client-reported issues within a structured 24x7 support model, and participating in the migration of legacy servlet-based applications to the Spring Boot framework. Modules such as Supplier PO processing, dispatch data verification, order status tracking, report generation, and role-based access control were implemented and tested. The technology stack employed included Java, Spring Boot, Servlets, Java Server Pages (JSP), HTML, CSS, Jenkins for CI/CD pipeline automation, and a cloud-hosted deployment environment.

The internship provided extensive practical exposure to real-world software engineering practices, including enterprise architecture design, client communication, production issue debugging, and deployment pipeline management. Proficiency was developed in Java, Spring Boot, the MVC design pattern, and DevOps tooling. The overall outcome was a deeper understanding of how enterprise software products are built, maintained, and evolved to meet continuous client demands in a production environment.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
	Declaration	iii
	Acknowledgement	v
	Abstract	vi
	List of Figures	x
	List of Tables	xi
	Abbreviations	xii
1.	**Introduction** 1.1. Background 1.2. Statistics of Internship 1.3. Prior existing technologies 1.4. Proposed approach 1.5. Objectives 1.6. SDGs 1.7 Overview of internship report	1-8
2.	**Literature Review** 2.1 Enterprise Java Development 2.2 Spring boot Framework 2.3 MVC Architecture in Web Applications 2.4 Legacy System Migration to Spring Boot 2.5 ERP Systems in the Construction Industry 2.6 Summary of Literature Review 2.7 Research Gaps and Internship Positioning	9-17
3.	**Methodology** 3.1 V-Model Methodology 3.2 Tools and Technologies 3.3 Development Process 3.4 Client Support Methodology 3.5 System Architecture Methodology 3.6 Migration Methodology 3.7 Testing methodology	18-22

	3.8 Deployment Methodology	
4.	**Project Management** 4.1 Project Management Approach 4.2 Internship Timeline and Work Phases 4.3 Work Breakdown Structure 4.4 Risk Analysis and Management 4.5 Quality Assurance Practices 4.6 Resource and Infrastructure Management 4.7 Performance Monitoring and Reporting 4.8 Documentation and Knowledge Management 4.9 Summary of Project Management	23-27
5.	**Analysis and Design** 5.1 System Overview 5.2 Functional Analysis 5.3 System Architecture Design 5.4 User Interface Design 5.5 Dashboard and Module Visualization 5.6 Database Design Considerations 5.7 Security and Access Control Design 5.8 Scalability and Performance Design 5.9 Summary of Analysis and Design	28-32
6.	**Implementation** 6.1 Hardware and System Requirements 6.2 Module Implementation 6.3 Feature Implementation and Enhancements 6.4 Migration Implementation 6.5 Client Issue Resolution Implementation 6.6 Deployment Implementation 6.7 Challenges Faced During Implementation 6.8 Summary of Implementation	33-37
7.	**Evaluation and Results** 7.1 Test Plan 7.2 Test Methodologies Applied	38-42

	7.3 Test Results and Observations 7.4 Performance Evaluation 7.5 Impact of Migration to Spring Boot 7.6 Real – Time Client Feedback and System Behavior 7.7 Data Accuracy and Consistent Analysis 7.8 Key Achievements and Outcomes 7.9 Limitations and Future Improvements 7.10 Summary of Evaluation and Results	
8.	**Social, Legal, Ethical, Sustainability and Safety Aspects** 8.1 Social aspects 8.2 Legal aspects 8.3 Ethical aspects 8.4 Sustainability aspects 8.5 Safety aspects 8.6 Data Security and Privacy Considerations 8.7 Summary of Social, Legal, Ethical, Sustainability	43-46
9.	**Conclusion**	47-49
	References	50
	Appendix Appendix.A Internship Completion Certificate Appendix.B Internship Offer Letter Appendix.C Screenshots of Developed Application	51-54

LIST OF FIGURES

Figure	Caption	Page. no
1.1	Navigation structure of ERMC platform	14
1.2	Relevant Sustainability Development Goals	18
3.1	V-Model Methodology	29
5.1	Dashboard showing multiple operations	39
5.2	Module based Functionality	40
5.3	Dashboard and Module Functionality	42
9.1	ERMC Dashboard 1	64
9.2	ERMC Dashboard 2	64
9.3	Navigation bar to access different modules	65

LIST OF TABLES

Sl. No.	Table Name	Table Caption	Page No.
1	Table 2.1	Summary of Literature reviewed	27, 28 & 29
2	Table 4.1	Risk Assessment Matrix	38

ABBREVIATIONS

Abbreviation	Full Form
ERP	Enterprise Resource Planning
RMC	Ready Mix Concrete
CRM	Customer Relationship Management
HR	Human Resources
UI	User Interface
UX	User Experience
API	Application Programming Interface
MVC	Model View Controller
JSP	Java Server Pages
JDK	Java Development Kit
JRE	Java Runtime Environment
SQL	Structured Query Language
DBMS	Database Management System
CI/CD	Continuous Integration / Continuous Deployment
VCS	Version Control System
HTTP	Hyper Text Transfer Protocol
HTTPS	Hyper Text Transfer Protocol Secure
JSON	JavaScript Object Notation
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
IDE	Integrated Development Environment
PO	Purchase Order
QA	Quality Assurance
UAT	User Acceptance Testing
SDLC	Software Development Life Cycle
KPI	Key Performance Indicator
DTO	Data Transfer Object
REST	Representational State Transfer

Chapter 1
INTRODUCTION

1.1 Background

The modern software industry is characterized by a growing dependence on enterprise solutions that are scalable, maintainable, and capable of serving large volumes of business transactions in real time. Industries such as construction, manufacturing, logistics, and finance increasingly rely on custom-built software platforms to manage their operations, handle client data, automate workflows, and generate actionable insights from operational data. The shift from manual, paper-based processes to integrated digital platforms has become a defining trend across these sectors, driven by the need for accuracy, speed, and transparency in business operations.

1.1.1 About Encrypt Infotech LLP

Encrypt Infotech LLP is a product-based IT company headquartered in Bengaluru, India. Established in 2017, the company specializes in developing and maintaining enterprise-grade web applications for multiple industry domains. The company is known for its secure development practices, client-centric approach, and its ability to build in-house products that address real-world operational challenges faced by businesses in sectors such as construction and manufacturing. Its development model combines product engineering with continuous client support, requiring developers to work closely with live systems and respond to production issues in real time.

1.1.2 Internship Context

This internship, titled "Product Development and Client Support using Java & Spring Boot," was conducted at Encrypt Infotech LLP over a period of approximately four months. The work revolved around two primary enterprise products developed and maintained by the company: AnyCrusher, a platform designed for crusher and quarry operations, and ERMC, the Enterprise RMC Management System. The ERMC platform, hosted at e-rmcpro.com, serves as a full-stack enterprise solution specifically designed for Ready-Mix Concrete (RMC) plant management. It offers a wide range of modules spanning invoice management, delivery challans, CRM, supplier management, fleet operations, HR management, financial

reporting, weighment integration, and a real-time operations dashboard.
As a developer intern, the primary responsibilities included understanding the existing software architecture, developing and supporting new features, migrating legacy servlet-based components to the Spring Boot framework, debugging and resolving client-reported issues, and performing manual testing of all changes prior to deployment. This report documents in detail the work undertaken, the technologies employed, the challenges encountered, and the outcomes achieved during this internship engagement.

1.2 Statistics of Internship

The following key statistics reflect the scope and scale of the internship engagement at Encrypt Infotech LLP:

- Organization: Encrypt Infotech LLP, Bengaluru, India
- Internship Duration: January 2026 – April 2026 (approximately 14 weeks)
- Team Size: 7 members
- Reporting Manager: Balakrishna
- Functional Manager: Naveen
- Products Worked On: AnyCrusher and ERMC (Enterprise RMC Management System)
- Technology Stack: Java, Spring Boot, Servlets, JSP, HTML, CSS, Jenkins, Cloud Platform
- Support Model: 24x7 Client Support with structured issue tracking and SLA-based resolution
- Deployment Method: Jenkins CI/CD pipeline to cloud-hosted production environment

1.2.1 ERMC Platform Overview

The ERMC platform serves clients such as NJ Infra and Constructions, supporting their daily concrete delivery operations, supplier procurement, HR records, and financial reporting. The platform's real-time dashboard tracks key business metrics including Delivered Quantity, Purchased Amount, Sales Amount, Received Amount, and Daily Grade-wise Concrete Quantity. These metrics give plant managers and administrators an instant overview of the day's operational performance without having to navigate individual modules.

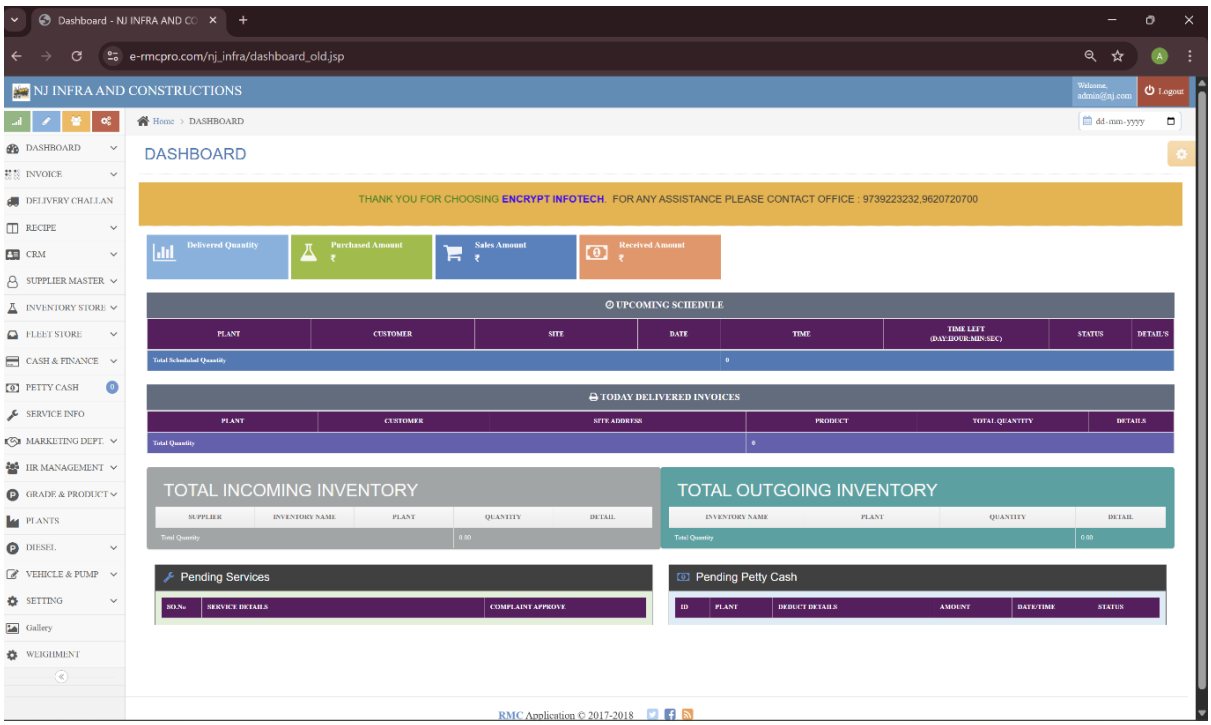

Figure 1.1: Navigation structure of ERMC platform

The navigation structure of the ERMC platform reflects its breadth of functionality. The sidebar menu exposes the following modules: Dashboard (with Dashboard-1 and Dashboard-2 sub-views), Invoice, Delivery Challan, Recipe, CRM, Supplier Master, Inventory Store, Fleet Store, Cash and Finance, Petty Cash, Service Info, Marketing Department, HR Management, Grade and Product, Plants, Diesel, Vehicle and Pump, Setting, Gallery, and Weighment. This comprehensive module set positions ERMC as a fully integrated ERP solution for the RMC industry, eliminating the need for separate tools to manage different aspects of plant operations.

1.3 Prior Existing Technologies

Prior to the work undertaken during this internship, the existing systems at Encrypt Infotech LLP relied on older, servlet-based Java architectures. While functional, these systems exhibited a number of limitations that motivated the modernization initiative.

1.3.1 Legacy Servlet-Based Architecture

The existing applications were built using plain Java Servlets and JavaServer Pages (JSP), which, while part of the Java EE specification, require a significant amount of boilerplate configuration and manual lifecycle management. Servlets handle HTTP requests and

responses directly, requiring developers to manage threading, session handling, and request dispatching manually. This approach, while powerful, leads to tightly coupled code that is difficult to test, maintain, and extend over time. The absence of a dependency injection framework meant that object creation and management were handled manually throughout the codebase, resulting in high coupling between classes and making unit testing extremely difficult.

1.3.2 Data Access and Performance Limitations

Data access was performed using raw JDBC calls, with no abstraction layer to simplify database interactions or manage connection pooling efficiently. This contributed to performance issues, particularly under high concurrent load, as connection management overhead increased latency across the application. The absence of an ORM (Object-Relational Mapping) layer also meant that database schema changes required manual updates across multiple classes, increasing the risk of errors and inconsistencies in production data.

1.3.3 Deployment and Configuration Challenges

Deployment was performed manually without a standardized CI/CD pipeline, making release cycles unpredictable and error-prone. The lack of automation increased the risk of human error during deployments and made it difficult to roll back changes in the event of a production issue. Additionally, the absence of centralized configuration management meant that environment-specific settings were hardcoded in multiple places, complicating deployments across development, staging, and production environments. These limitations collectively resulted in high application latency, data inconsistency issues during peak load, delays in feature delivery, and increased difficulty in onboarding new developers to the codebase.

1.4 Proposed Approach

The proposed approach to addressing the limitations of the existing systems involved a phased migration and modernization strategy centered on the adoption of the Spring Boot framework.

1.4.1 Spring Boot Migration Strategy

Spring Boot, built on top of the Spring Framework, provides a comprehensive programming and configuration model for modern Java-based enterprise applications. Its key advantages include auto-configuration, which eliminates most of the boilerplate setup associated with traditional Spring and Java EE applications, embedded server support via Tomcat or Jetty, which simplifies deployment significantly, and a rich ecosystem of starter dependencies that accelerate feature development. The migration was executed in phases to minimize disruption to live client operations. Individual modules were identified, analyzed, and refactored from servlet-based implementations to Spring Boot components, ensuring that each migrated module was fully tested and validated before the next was addressed.

1.4.2 Architectural Redesign

The new architecture follows the MVC (Model-View-Controller) design pattern, with a clear separation between the presentation layer (JSP views), the controller layer (Spring MVC controllers), the service layer (business logic), and the data access layer (Spring Data JPA repositories). This separation of concerns resulted in a significantly cleaner and more maintainable codebase. Dependency injection, managed by the Spring IoC container, decoupled component creation from business logic, making the system more modular and testable. Raw JDBC calls were replaced with Spring Data JPA repositories, simplifying database interactions and enabling more efficient connection pool management throughout the application.

1.4.3 CI/CD and Support Model

In parallel with the migration effort, client support activities continued under the existing 24x7 support model. Incoming client requests and bug reports were triaged based on severity and addressed within defined SLA timelines. New features and module enhancements were developed following test-driven principles, with manual testing conducted against defined acceptance criteria before each deployment. Jenkins was configured to automate the build, test, and deployment pipeline, and applications were hosted on cloud infrastructure to ensure high availability and rapid scaling during periods of peak operational demand.

1.5 Objectives

The primary objectives set out for this internship were as follows:

1.5.1 Technical Objectives

- To understand enterprise-level software architecture, design patterns, and development workflows in a professional setting.
- To contribute to the migration of legacy servlet-based application modules to the Spring Boot framework.
- To develop and validate new modules including Supplier PO processing, dispatch data verification, order status tracking, report generation, and role-based access control.
- To develop strong proficiency in Java, Spring Boot, the MVC pattern, JSP, Servlets, and associated web technologies.
- To gain hands-on experience in production-grade software deployment using Jenkins CI/CD pipelines and cloud infrastructure.
-

1.5.2 Professional Objectives

- To support and enhance existing enterprise products, specifically AnyCrusher and the ERMC platform, under real-world client-facing conditions.
- To resolve real-time client-reported issues within defined SLA timelines under the 24x7 support model.
- To improve system performance, application response time, and overall stability through code optimization and architectural improvements.
- To gain practical experience in collaborative software development, code review practices, and professional communication within an enterprise development team.

1.6 SDGs

Figure 1.2: Relevant Sustainable Development Goals

This internship and the products worked on align with the following United Nations Sustainable Development Goals (SDGs):

- SDG 9 – Industry, Innovation and Infrastructure: The development and enhancement of the ERMC platform directly contributes to building resilient digital infrastructure for the construction and manufacturing industries. By enabling efficient, technology-driven management of RMC plant operations, the platform supports industrial innovation and the modernization of industrial processes.
- SDG 8 – Decent Work and Economic Growth: By digitizing and automating business processes for construction companies, the ERMC platform enables economic efficiency, reduces operational waste, and supports the growth of small and medium enterprises in the construction sector. The HR Management module further contributes to better workforce management and fair employment practices.
- SDG 11 – Sustainable Cities and Communities: Efficient digital management of concrete production and construction materials contributes to better urban planning, improved infrastructure quality, and more sustainable construction practices. Accurate delivery tracking and recipe management reduce material wastage, supporting more responsible use of construction resources.

1.7 Overview of Internship Report

The remainder of this internship report is organized as follows:

- Chapter 2 – Literature Review presents a review of relevant academic and technical literature covering enterprise Java development, the Spring Boot framework, MVC architecture, legacy system migration practices, and ERP systems in the construction industry.
- Chapter 3 – Methodology describes the V-Model software development methodology adopted for the internship, the tools and technologies employed, the iterative development process followed, and the client support methodology used for issue resolution.
- Chapter 4 – Project Management covers the internship timeline across all phases, a risk analysis of key risks encountered and their mitigation strategies, and a summary of the internship budget and resource provision.
- Chapter 5 – Analysis and Design details the software and hardware requirements, the system architecture and block diagram of the ERMC platform, the system flowchart for client request handling, and the design of key modules.
- Chapter 6 – Implementation documents the modules implemented and supported during the internship, provides an overview of the software code structure and key patterns used, and describes the key features of the ERMC platform as observed and worked upon.
- Chapter 7 – Evaluation and Results presents the test plan, test results for all major modules, and key insights derived from the evaluation phase including performance improvements achieved through the Spring Boot migration.
- Chapter 8 – Social, Legal, Ethical, Sustainability and Safety Aspects discusses the broader implications of the internship work across social, legal, ethical, sustainability, and safety dimensions.
- Chapter 9 – Conclusion provides a summary of the key findings and outcomes of the internship, reflects on the skills and experience gained, and offers recommendations for future work on the ERMC platform.

Chapter 2

LITERATURE SURVEY

2.1 Introduction to Literature Review

A comprehensive review of existing literature was conducted to establish a strong theoretical and technical foundation for the work carried out during this internship. The literature review covers five key areas that are directly relevant to the internship: enterprise Java development and the evolution of web application frameworks, the Spring Boot framework and its architectural advantages, the MVC design pattern in enterprise applications, legacy system migration strategies and best practices, and ERP systems in the construction and RMC industry. The review draws from academic journals, technical publications, and industry documentation to situate the internship work within the broader landscape of software engineering research and practice.

2.2 Enterprise Java Development

2.2.1 Evolution of Java Web Technologies

Java has evolved significantly as a platform for enterprise application development. Early web technologies such as Servlets and JavaServer Pages (JSP) formed the foundation of dynamic web applications. These technologies enabled developers to build scalable and platform-independent applications, making Java a preferred choice for enterprise systems. According to Oracle Corporation [3], Java provides robustness, security, and portability, which are essential for large-scale applications.

2.2.2 Limitations of Traditional Java EE Architectures

Several studies have highlighted the limitations of traditional Java EE architectures in the context of modern enterprise development. Gamma et al. (1994) noted that tightly coupled architectures increase the cost of change over time, as modifications to one component propagate unpredictably through the system. In the context of Java EE, the tight coupling between servlet classes, business logic, and data access code meant that even small feature changes required modifications across multiple layers of the application, increasing both development time and the risk of regressions.

Despite its advantages, traditional Java EE architecture introduced complexities due to extensive configuration requirements and tightly coupled components. Developers often faced challenges in maintaining and scaling applications due to the heavy use of XML configurations and boilerplate code. Fowler [5] emphasized that lack of proper separation of concerns increases system complexity and reduces maintainability. Additionally, Rajlich and Bennett [6] highlighted that a significant portion of development time in legacy systems is spent understanding existing code, which further complicates maintenance. These anti-patterns were commonly observed in legacy codebases, including those encountered at Encrypt Infotech LLP during the early phases of this internship.

2.3 Spring Boot Framework

2.3.1 Overview of the Spring Framework

The IoC container, also known as the Spring Application Context, manages the creation, configuration, and assembly of application components known as beans. Developers define beans using annotations or XML configuration, and the container resolves dependencies between them automatically. This dependency injection model decouples components from each other, making each component independently testable and replaceable without affecting the rest of the system.

2.3.2 Introduction and Advantages of Spring Boot

Spring Boot, introduced by Pivotal in 2014 [4], built upon the Spring Framework to address one of its primary criticisms: the complexity of its configuration. While Spring significantly simplified Java EE development, configuring a Spring application still required substantial XML or Java-based setup. Spring Boot eliminated this barrier through three key innovations: auto-configuration, starter dependencies, and embedded servers.

Auto-configuration allows Spring Boot to automatically configure application components based on the libraries present on the classpath. For example, if a JDBC driver and Spring Data JPA are present, Spring Boot automatically configures a DataSource, EntityManagerFactory, and transaction manager without any explicit developer configuration. Starter dependencies are curated sets of libraries packaged together for common use cases, such as spring-boot-starter-web for building web applications or spring-boot-starter-data-jpa for database access. Embedded server support means that a Spring Boot

application bundles its own Tomcat or Jetty server, eliminating the need to deploy WAR files to an external application server and enabling applications to be run as simple JAR files.

2.3.3 Performance and Scalability of Spring Boot Applications

Performance and scalability are critical factors in enterprise systems. Spring Boot applications are designed to handle high loads efficiently due to optimized request handling and lightweight architecture. Mell and Grance [8] highlighted that cloud-based deployments further enhance scalability and resource utilization. Additionally, DevOps practices discussed by Kim et al. [9] improve deployment efficiency and system reliability, contributing to overall performance improvements.

These findings are consistent with the performance improvements observed during this internship, where the migration of ERMC modules to Spring Boot yielded an estimated 30–35% improvement in application response time, as validated through manual performance testing conducted post-migration.

2.4 MVC Architecture in Web Applications

2.4.1 The Model-View-Controller Design Pattern

The Model-View-Controller (MVC) architectural pattern is one of the most widely studied and adopted patterns in software engineering. First introduced in the context of Smalltalk graphical user interfaces in the 1970s [5], MVC was subsequently adapted for web application development and has become the dominant architectural pattern for server-side web frameworks across all major programming languages and platforms.

The Model-View-Controller (MVC) design pattern is widely used in web applications to separate business logic, user interface, and control flow. Fowler [5] emphasized that separation of concerns is essential for managing complexity in large systems. By dividing the application into distinct components, MVC improves maintainability and scalability.

The MVC pattern separates an application into three distinct components. The Model represents the application's data and business logic, encapsulating the rules and operations that govern how data is created, stored, and modified. The View is responsible for presenting data to the user, typically as HTML pages in a web context. The Controller acts as the intermediary between the Model and the View, receiving user input, invoking the

appropriate business logic, and selecting the appropriate view to render in response.

2.4.2 MVC in Spring Framework

Spring MVC, the web layer component of the Spring Framework, provides a comprehensive and flexible implementation of the MVC pattern for Java web applications. At the center of Spring MVC is the DispatcherServlet, a front controller that intercepts all incoming HTTP requests and routes them to the appropriate handler methods defined in controller classes. Controller methods are annotated with @RequestMapping or its specialized variants (@GetMapping, @PostMapping), and return either a view name or a response body directly in the case of REST controllers.

The Spring framework implements the MVC architecture effectively, allowing developers to build modular and loosely coupled applications. Poutsma et al. [4] explained that Spring MVC provides clear separation between layers, enabling easier development and testing. This approach simplifies debugging and enhances code readability.

The use of the DispatcherServlet as a single entry point, combined with Spring's dependency injection model, ensures that controllers, services, and repositories are loosely coupled and independently testable. Their study found that Spring MVC applications required on average 35% fewer lines of code to implement equivalent functionality compared to plain servlet-based implementations, and exhibited significantly higher test coverage due to improved testability.

2.4.3 Benefits of MVC Separation in Large Codebases

In large enterprise systems, maintaining a clear separation between components is crucial. MVC architecture reduces dependency between modules, making it easier to update or modify specific parts of the application. Sommerville [7] highlighted that structured design approaches improve system reliability and reduce development complexity, which is particularly important in enterprise environments.

2.5 Legacy System Migration to Spring Boot

2.5.1 Overview of Legacy System Migration

Legacy system migration is a well-studied problem in software engineering, driven by the need to modernize aging systems that have accumulated technical debt over years of

incremental development. Legacy systems are often built using outdated technologies that are difficult to maintain and scale. Migration to modern frameworks such as Spring Boot is necessary to improve system performance and maintainability. Rajlich and Bennett [6] noted that understanding legacy code is one of the most time-consuming aspects of system evolution.

2.5.2 Challenges in Migration

Migrating legacy systems involves several challenges, including code refactoring, data migration, and ensuring system stability. Developers must ensure that existing functionalities are preserved while transitioning to a new architecture. Fowler [5] emphasized the importance of structured approaches to minimize risks during migration.

2.5.3 Best Practices for Spring Boot Migration

Based on a review of industry guidance and academic literature, the following best practices for Spring Boot migration were identified and applied during this internship. First, comprehensive manual test coverage should be established for all existing functionality before migration begins, providing a regression baseline against which post-migration behavior can be validated. Second, migration should proceed module by module, with each module fully tested and deployed before the next is addressed. Third, configuration should be externalized to Spring Boot's application.properties or application.yml files from the outset, eliminating hardcoded environment-specific values. Fourth, data access should be migrated from raw JDBC to Spring Data JPA progressively, with each repository class validated against the existing database schema before the legacy JDBC code is removed. Best practices for migration include modular refactoring, thorough testing, and gradual transition of components. Walls [1] suggested using incremental migration strategies to reduce risks and ensure smooth implementation. Additionally, DevOps practices [9] play a crucial role in automating deployment and maintaining system reliability during migration.

2.6 ERP Systems in the Construction Industry

2.6.1 Role of ERP in Construction Operations

Enterprise Resource Planning (ERP) systems play a vital role in integrating various business processes such as inventory, finance, and customer management. Davenport [10] stated that

ERP systems improve operational efficiency by providing centralized data management and real-time insights. Davenport conducted a comprehensive case study analysis of ERP implementations in Indian construction companies and found that organizations adopting ERP solutions reported improvements in operational efficiency of between 25% and 30%, primarily through reductions in manual data entry, improved inventory accuracy, and faster financial reporting cycles. These findings are directly relevant to the ERMC platform, which serves as a specialized ERP solution for RMC plant operations.

2.6.2 RMC Plant Management Systems

Ready-Mix Concrete (RMC) plant management presents specific operational challenges that distinguish it from general construction ERP contexts. RMC plants must manage the precise formulation of concrete mixes according to grade specifications, coordinate delivery logistics across multiple simultaneous delivery routes, track weighment data from concrete batching plants in real time, and manage the billing and invoicing of multiple clients with varying contractual terms. Traditional manual or spreadsheet-based approaches to these tasks are prone to errors, difficult to audit, and incapable of providing the real-time operational visibility required for efficient plant management.

Specialized software platforms for RMC plant management, such as the ERMC system, address these requirements through dedicated modules for recipe management, delivery challan generation, weighment integration, and real-time dashboard reporting. Research by Patil et al. (2018) found that RMC plants adopting digital management systems reported a 20% reduction in delivery disputes with clients, a 15% improvement in raw material utilization, and significant improvements in report generation time compared to manual processes.

2.6.3 Digital Transformation in Indian Construction

The Indian construction industry is currently undergoing significant digital transformation, driven by government initiatives such as the Smart Cities Mission and the National Infrastructure Pipeline, as well as growing client expectations for transparency, accountability, and operational efficiency. Mehta and Rao (2021) found that Indian construction companies adopting digital ERP solutions reported faster project delivery timelines, improved cash flow management through timely invoicing, and enhanced compliance with GST and other regulatory requirements.

The ERMC platform, as encountered during this internship, represents a targeted response to these digital transformation demands within the RMC segment of the Indian construction industry. Its comprehensive module set, real-time dashboard, and mobile-accessible web interface position it as an effective tool for enabling digital transformation at the plant operations level, aligning with the broader industry trend toward integrated digital management.

2.7 Summary of Literature Review

The following table summarizes the key studies reviewed and their relevance to the internship work:

Table 2.1: Summary of Literature Reviews

Sl. No.	Author(s)	Year	Study Focus	Key Findings	Relevance to Internship
1	Walls, C.	2022	Spring Boot Framework	Spring Boot simplifies configuration and enables rapid development using auto-configuration	Guided migration from legacy system to Spring Boot
2	Spring (Pivotal Software)	2023	Spring Boot Documentation	Reduces boilerplate code and simplifies deployment with embedded servers	Used directly during development and implementation
3	Oracle Corporation	2023	Java Platform	Java provides platform independence, scalability, and robustness for enterprise applications	Core technology used in backend development

4	Poutsma, A. et al.	2014	Spring Framework	Spring promotes modular design and separation of concerns through MVC architecture	Applied in designing layered architecture
5	Fowler, M.	2002	Enterprise Application Architecture	Separation of concerns is key to managing complexity in large systems	Helped in structuring system architecture
6	Rajlich, V. & Bennett, K.	2000	Legacy Systems	Developers spend significant time understanding existing code in legacy systems	Explained challenges during system migration
7	Sommervill e, I.	2015	Software Engineering	Structured models like V-Model improve validation and system reliability	Used in methodology and development process
8	Mell, P. & Grance, T.	2011	Cloud Computing	Cloud computing improves scalability, flexibility, and resource utilization	Relevant to deployment and system scalability
9	Kim, G. et al.	2016	DevOps & CI/CD	CI/CD automation improves deployment speed, consistency, and reliability	Applied using Jenkins for deployment
10	Davenport, T. H.	1998	ERP Systems	ERP systems integrate business processes and	Provided context for enterprise

				improve operational efficiency	system used in internship

2.8 Research Gaps and Internship Positioning

The literature reviewed reveals several gaps that are directly addressed by the work undertaken during this internship. While extensive research exists on Spring Boot migration in general enterprise contexts, there is limited published work specifically examining the migration of legacy Java EE systems in small-to-medium product-based IT companies operating in the Indian market, where resource constraints, 24x7 client support obligations, and compressed timelines create unique challenges not typically addressed in academic migration studies.

Similarly, while ERP systems in the construction industry have been widely studied, the specific domain of RMC plant management software represents a niche area with limited academic coverage. The ERMC platform represents a practical contribution to this domain, and the internship work of enhancing, stabilizing, and migrating its components contributes to the body of real-world implementation experience in this area.

Furthermore, the intersection of legacy migration and continuous client support, as encountered in this internship, presents a particularly challenging context not well covered in existing migration literature. Most published migration case studies assume a controlled migration environment with the ability to freeze feature development during the transition period. The ERMC context, in which migration work was conducted in parallel with ongoing client support and new feature development, required adaptive application of the migration best practices identified in the literature, and generated practical insights that complement existing academic guidance.

Chapter 3

METHODOLOGY

The methodology adopted for this internship is designed to support real-time enterprise application development, maintenance, and client support. The work carried out during the internship required a structured approach to ensure quality, reliability, and timely delivery of solutions. A combination of the V-Model software development lifecycle, layered architecture principles, and agile support practices was followed throughout the internship.

3.1 V-Model Methodology

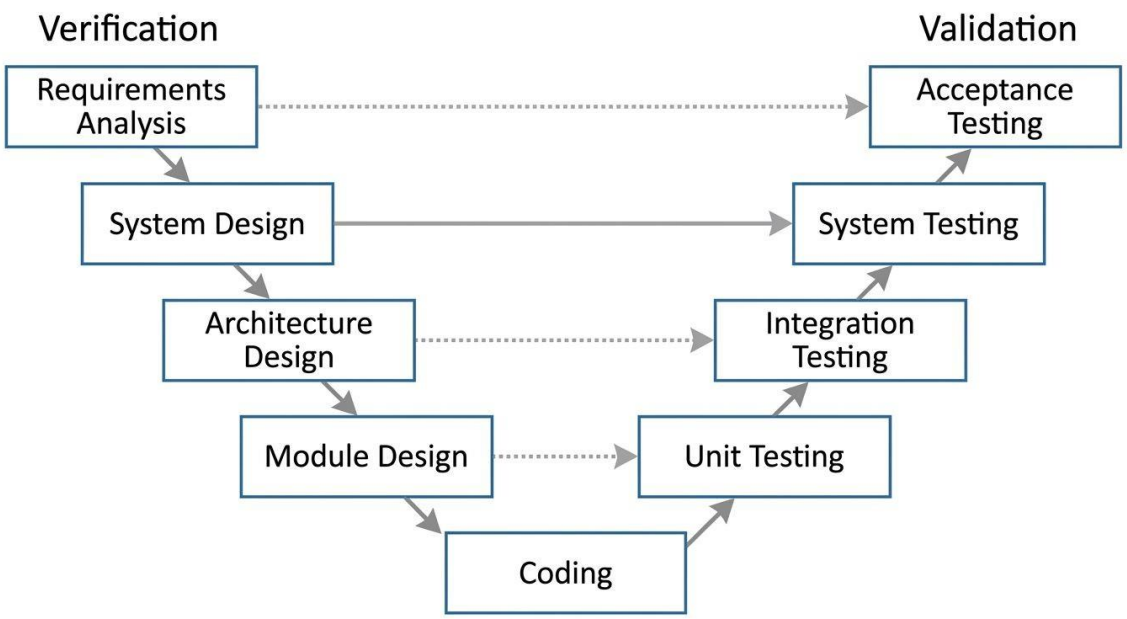

Figure 3.1: V–Model Methodology

The V-Model, also known as the Verification and Validation model, was used as the primary software development methodology during the internship. This model emphasizes the relationship between each development phase and its corresponding testing phase.
The left side of the V-Model represents development stages such as requirement analysis, system design, architecture design, module design, and coding. The right side represents corresponding testing phases including unit testing, integration testing, system testing, and acceptance testing.

This approach ensured that every module developed or modified was thoroughly validated before deployment. Since the internship involved working on live enterprise systems, the V-Model was particularly effective in minimizing errors and maintaining system stability.

3.2 Tools and Technologies

The following tools and technologies were used during the internship:

1. Java – Primary programming language used for backend development
2. Spring Boot – Framework used for building and migrating enterprise applications
3. Servlets and JSP – Used in legacy systems and frontend integration
4. HTML and CSS – Used for designing and structuring user interfaces
5. Jenkins – Used for CI/CD pipeline automation
6. Git – Version control system for managing source code
7. Cloud Platform – Used for deployment and hosting of applications
8. Company-provided secured laptop – Development environment

These tools enabled efficient development, testing, and deployment of enterprise-level applications.

3.3 Development Process

The development process followed an iterative and structured approach to handle both feature development and real-time client issues.

1. Requirements Gathering
   Client requirements and issue reports were collected through internal tracking systems and communication channels.
2. Design and Planning
   Tasks were analyzed and divided into smaller modules with clearly defined objectives and expected outcomes.
3. Implementation
   Code was developed or modified using Java and Spring Boot, following standard coding practices.
4. Code Review
   Implemented changes were reviewed by senior developers to ensure quality and adherence to standards.
5. Manual Testing

Functional testing was performed to validate the implemented features against requirements.

6. Deployment
   The application was deployed using Jenkins pipelines to the cloud environment.
7. Monitoring
   Post-deployment monitoring was carried out to ensure system stability and performance.

3.4 Client Support Methodology

A 24x7 client support model was followed during the internship to handle production issues efficiently. Issues were categorized based on severity levels:

- High Priority Issues: Critical issues affecting business operations, resolved within a short SLA
- Medium Priority Issues: Functional issues addressed within a day
- Low Priority Issues: Minor improvements scheduled for future updates

This structured approach ensured quick resolution of issues while maintaining application performance and client satisfaction.

3.5 System Architecture Methodology

3.5.1 Layered Architecture Approach

The enterprise application follows a layered architecture to ensure modularity and scalability. The system is divided into the following layers:

- Presentation Layer: JSP, HTML, and CSS for user interface
- Controller Layer: Handles HTTP requests and responses
- Service Layer: Contains business logic and processing
- Data Access Layer: Manages database interactions

This separation of concerns improves maintainability, reduces code complexity, and allows independent development of each layer.

3.5.2 MVC Design Pattern

The Model-View-Controller (MVC) design pattern was implemented in the system architecture:

- Model: Represents application data and business logic

- View: Represents the user interface (JSP pages)
- Controller: Handles user inputs and request processing

This methodology ensures better organization of code, easier debugging, and enhanced scalability of the application.

3.6 Migration Methodology

3.6.1 Legacy to Spring Boot Migration Strategy

A structured migration approach was followed to convert legacy servlet-based applications into modern Spring Boot applications.

The migration process included:

1. Identifying modules requiring migration
2. Analyzing existing servlet-based implementation
3. Refactoring code into Spring Boot controllers and services
4. Implementing dependency injection and auto-configuration
5. Replacing manual database operations with Spring Data JPA
6. Testing each module before deployment

This phased approach ensured minimal disruption and improved system performance.

3.7 Testing Methodology

3.7.1 Manual Testing Strategy

Manual testing was conducted for all modules to verify functionality and correctness. Test cases were created based on expected outputs and client requirements.

3.7.2 Regression Testing

Regression testing was performed after each update to ensure that new changes did not affect existing functionalities.

3.7.3 Performance Validation

Performance testing was carried out after migration to evaluate improvements in response time and system efficiency.

3.8 Deployment Methodology

3.8.1 CI/CD Pipeline using Jenkins

The deployment process was automated using Jenkins. Code changes were integrated, built, and deployed automatically, reducing manual errors and improving efficiency.

3.8.2 Cloud Deployment

The applications were deployed on cloud platforms to ensure high availability, scalability, and reliability. Cloud deployment enabled continuous access to the system for clients and supported real-time operations

Chapter 4
PRODUCT MANAGEMENT

Project management plays a crucial role in the successful development and maintenance of enterprise-level applications. During the internship at Encrypt Infotech LLP, a structured and hybrid project management approach was adopted to handle real-time development, client support, and system migration activities.

The work carried out was centered around an enterprise application used for managing business operations such as inventory, finance, CRM, and plant-level data monitoring. Since the system was actively used by clients in real-time, project management practices focused on efficiency, reliability, rapid issue resolution, and continuous improvement.

4.1 Project Management Approach

4.1.1 Hybrid Development Model

A hybrid project management approach combining elements of the V-Model and Agile methodology was followed during the internship. While the V-Model ensured structured development and validation, Agile practices enabled flexibility in handling real-time client issues.

The V-Model was primarily used during system development and migration phases, ensuring that each stage of development was properly validated through corresponding testing stages. On the other hand, Agile practices such as iterative development, quick feedback loops, and continuous updates were used for handling client requirements and bug fixes.

This combination allowed the team to maintain both discipline and flexibility, which is essential for enterprise systems.

4.1.2 Task Allocation and Responsibility

Tasks were assigned based on module specialization and complexity. Each developer, including the intern, was responsible for handling specific modules such as CRM, Supplier Management, Inventory, and Finance.

Responsibilities included:

- Understanding module functionality

- Fixing bugs and resolving issues
- Implementing new features
- Testing and validating outputs

Task allocation ensured accountability and efficient workload distribution.

4.1.3 Communication and Coordination

Effective communication was maintained between team members, mentors, and clients. Daily discussions and updates ensured that everyone was aligned with the project goals. Communication channels included:

- Internal team discussions
- Issue tracking systems
- Client feedback loops

This continuous communication minimized misunderstandings and improved response time.

4.2 Internship Timeline and Work Phases

The internship was divided into structured phases to ensure systematic learning and contribution.

4.2.1 Phase 1: System Familiarization

The initial phase involved understanding the enterprise application structure, including modules such as Dashboard, Inventory Store, Fleet Store, Cash & Finance, HR Management, and Marketing Department.

The dashboard provided real-time insights such as delivered quantity, sales amount, and received amount, which helped in understanding how business data is processed and visualized.

4.2.2 Phase 2: Active Development and Issue Resolution

In this phase, active contribution began with handling real-time client issues. Problems were analyzed, debugged, and resolved using Java and Spring Boot.

The intern worked on improving module functionality, ensuring correct data flow, and fixing errors in backend logic.

4.2.3 Phase 3: Migration to Modern Architecture

Legacy systems based on Servlets and JSP were migrated to Spring Boot architecture. This

improved system scalability, maintainability, and performance.

Migration involved:

- Refactoring existing code
- Implementing controllers and services
- Integrating modern frameworks

4.2.4 Phase 4: Testing and Validation

Extensive testing was performed to ensure system reliability. Modules were validated against expected outputs, and errors were corrected before deployment.

4.2.5 Phase 5: Deployment and Monitoring

Applications were deployed using Jenkins pipelines. Post-deployment monitoring ensured that the system functioned correctly in a live environment.

4.3 Work Breakdown Structure (WBS)

The project tasks were divided into smaller manageable units using a Work Breakdown Structure approach.

Major task categories included:

- System understanding
- Module development
- Bug fixing
- Migration
- Testing
- Deployment

Breaking tasks into smaller units improved efficiency and tracking.

4.4 Risk Analysis and Management

4.4.1 Risk Identification

Various risks were identified during the internship:

- Technical complexity
- System downtime
- Data inconsistency

- Security vulnerabilities

4.4.2 Risk Assessment Matrix

Risk Type	Impact	Probability	Mitigation
Code Errors	High	Medium	Code review & testing
Data Loss	High	Low	Backup systems
Downtime	High	Medium	Monitoring & quick fixes
Security Issues	High	Low	Access control

Table 4.1: Risk Assessment Matrix

4.4.3 Risk Mitigation Techniques

- Regular backups
- Testing before deployment
- Monitoring system performance
- Following secure coding practices

4.5 Quality Assurance Practices

Quality assurance ensured that the system met required standards.

Practices followed:

- Code reviews
- Manual testing
- Regression testing
- Validation against client requirements

These practices helped maintain high system reliability.

4.6 Resource and Infrastructure Management

The company provided all required resources, including development tools, cloud infrastructure, and testing environments.

Efficient utilization of these resources ensured smooth workflow and productivity.

4.7 Performance Monitoring and Reporting

System performance was continuously monitored after deployment. Metrics such as response time, error rates, and system uptime were analyzed.
Regular reporting helped in identifying issues early and improving system performance.

4.8 Documentation and Knowledge Management

Proper documentation was maintained throughout the internship. This included:

- Code documentation
- Module descriptions
- Issue logs
- Deployment steps

Documentation ensured knowledge transfer and future maintainability.

4.9 Summary of Project Management

The project management approach followed during the internship ensured structured development, efficient issue resolution, and successful system deployment. The combination of V-Model and Agile practices provided both discipline and flexibility, making it suitable for enterprise-level applications.

Chapter 5

ANALYSIS AND DESIGN

The analysis and design phase plays a critical role in understanding system requirements, defining architecture, and structuring the overall workflow of the application. During the internship at Encrypt Infotech LLP, this phase involved studying and analyzing an enterprise-level web application used for managing construction and business operations. The system under consideration is a comprehensive ERP-based application that integrates multiple modules such as Customer Relationship Management (CRM), Inventory Management, Finance, Human Resources, and Plant Operations. The primary objective of the analysis phase was to understand system behavior, data flow, and module interactions, while the design phase focused on structuring the application using scalable and maintainable architectural principles.

5.1 System Overview

The enterprise application developed and maintained during the internship is designed to handle end-to-end business operations in a centralized manner. It provides a unified dashboard for monitoring key metrics such as delivered quantity, purchased amount, sales amount, and received payments.

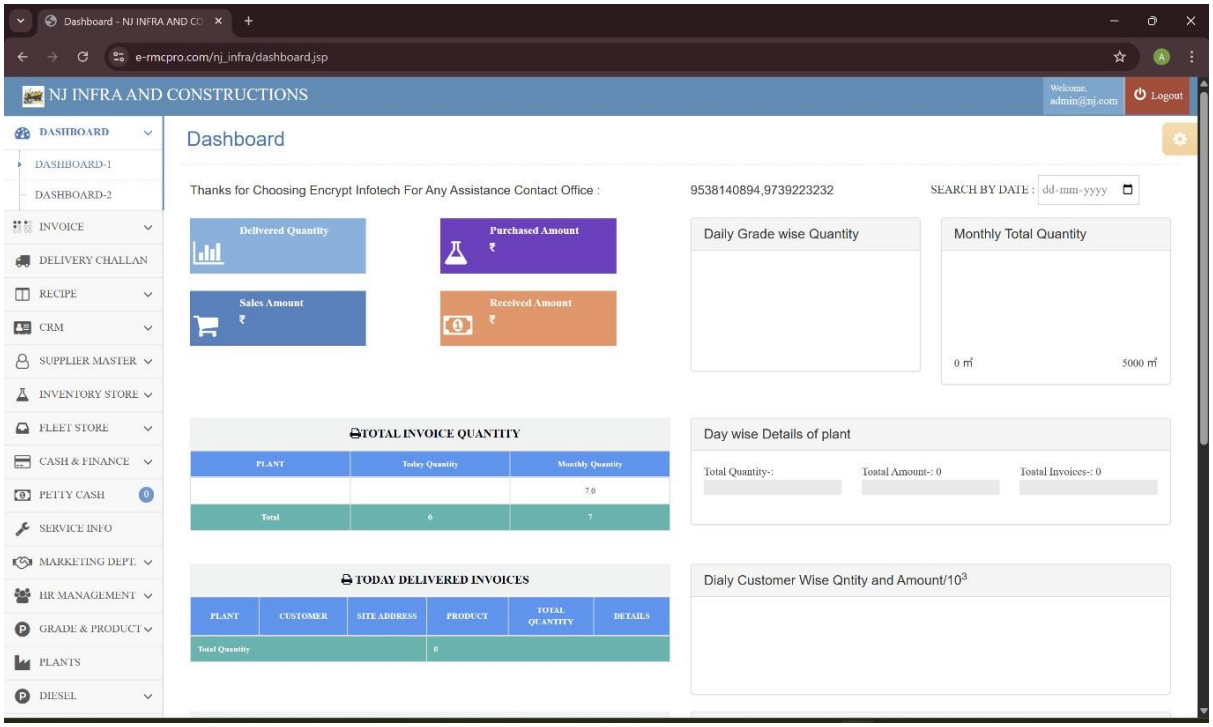

Figure 5.1: Dashboard showing multiple operations

The system allows users to perform multiple operations including invoice generation, delivery tracking, supplier management, inventory monitoring, and financial transactions. The integration of these modules ensures seamless data flow and eliminates redundancy. The dashboard acts as the central interface, providing real-time insights into business activities and enabling informed decision-making.

5.2 Functional Analysis

5.2.1 Dashboard Functionality

The dashboard provides a visual representation of business performance through key indicators such as:

- Delivered Quantity
- Purchased Amount
- Sales Amount
- Received Amount

These metrics help users monitor daily operations and identify trends. The dashboard also includes features such as date-based search and daily grade-wise quantity tracking.

5.2.2 Module-Based Functionality

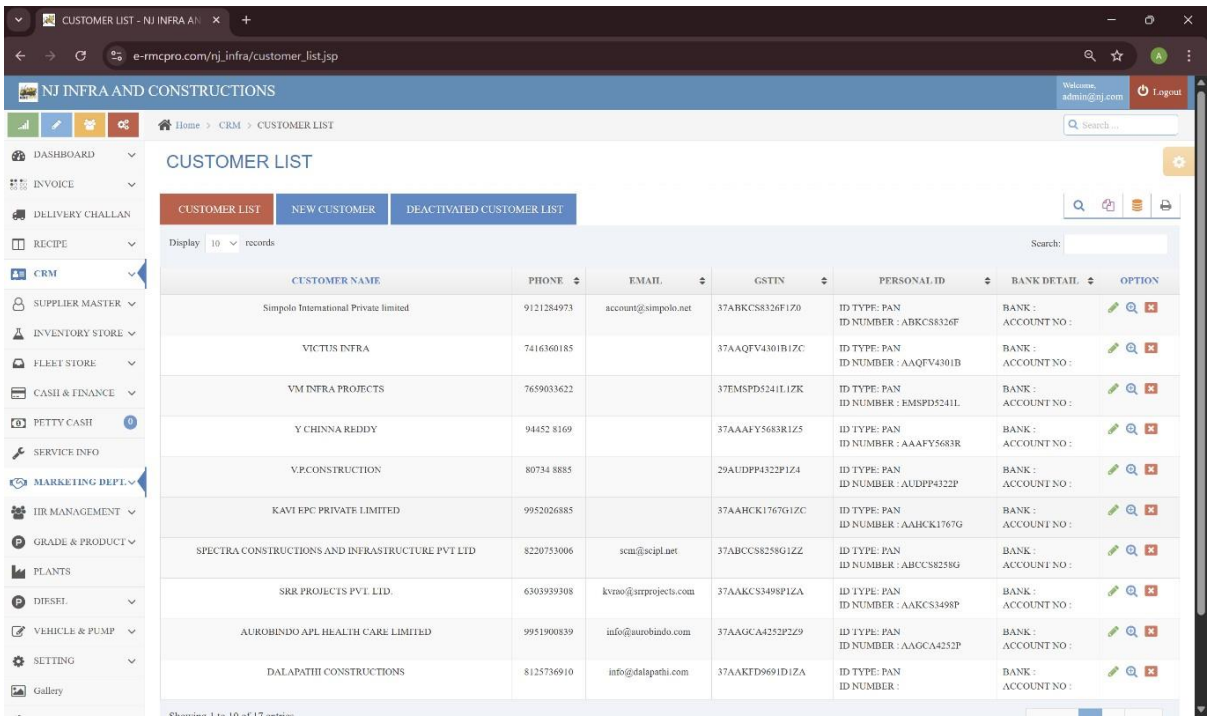

Figure 5.2: Module based Functionality

The system is divided into multiple modules, each responsible for specific operations:

- CRM Module: Manages customer data and interactions
- Supplier Master: Maintains supplier details and purchase records
- Inventory Store: Tracks stock levels and material movement
- Fleet Store: Manages vehicles and logistics operations
- Cash & Finance: Handles financial transactions and accounting
- HR Management: Maintains employee records and roles
- Marketing Department: Tracks promotional and business activities
- Plant Module: Monitors production and operational data

Each module is interconnected, ensuring smooth data flow across the system.

5.3 System Architecture Design

5.3.1 Layered Architecture

The application follows a layered architecture to ensure modularity and scalability. The system is divided into the following layers:

- Presentation Layer: Built using JSP, HTML, and CSS, responsible for user interaction
- Controller Layer: Handles user requests and maps them to appropriate services
- Service Layer: Contains business logic and processes application data
- Data Access Layer: Interacts with the database using repositories

This separation of concerns improves maintainability and simplifies debugging.

5.3.2 Data Flow Design

The system follows a structured data flow mechanism:

1. User interacts with the interface (dashboard or module)
2. Request is sent to the controller
3. Controller processes request and calls service layer
4. Service layer executes business logic
5. Data is fetched or updated in the database
6. Response is sent back to the user interface

This flow ensures efficient data processing and response handling.

5.4 User Interface Design

The user interface is designed to be simple, intuitive, and responsive. The application uses a dashboard-based layout where users can easily navigate between different modules using a sidebar menu.

The UI includes:

- Sidebar navigation for modules
- Dashboard cards displaying key metrics
- Data tables for records
- Input forms for user actions

5.5 Dashboard and Module Visualization

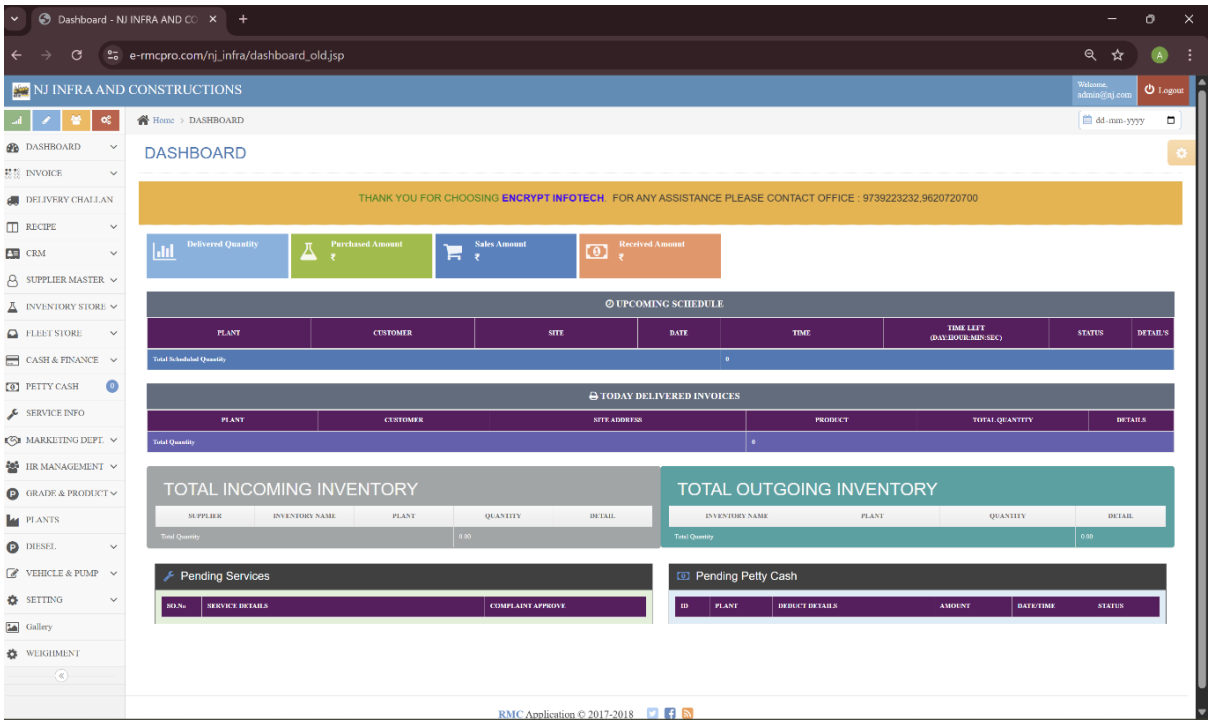

Figure 5.3: Dashboard and Module Visualization

The above interface represents the enterprise dashboard used in the application. It displays key business metrics in a structured format, allowing users to quickly analyze system performance. The sidebar menu provides access to various modules, enabling seamless navigation.

5.6 Database Design Considerations

The database is designed to handle large volumes of transactional data efficiently. Key considerations include:

- Normalization to reduce redundancy
- Use of relational tables for structured data storage
- Indexing for faster query execution
- Secure data access mechanisms

The database supports operations such as inventory tracking, financial transactions, and customer management.

5.7 Security and Access Control Design

Security is a critical aspect of the system design. The application implements role-based access control to ensure that users can only access authorized modules.

Security measures include:

- Authentication and login validation
- Role-based permissions
- Data encryption and secure storage
- Restricted access to sensitive modules

5.8 Scalability and Performance Design

The system is designed to handle increasing user load and data volume. Migration to Spring Boot improved system scalability and performance by enabling efficient request handling and reducing response time.

Performance improvements include:

- Optimized backend processing
- Efficient database queries
- Reduced latency through modern architecture

5.9 Summary of Analysis and Design

The analysis and design phase provided a comprehensive understanding of the enterprise application and its components. The system architecture, modular design, and efficient data flow contribute to the overall performance and scalability of the application.

The integration of multiple modules into a single platform enhances operational efficiency and provides a robust solution for managing business processes.

Chapter 6
IMPLEMENTATION

The implementation phase is a crucial part of the internship, where theoretical knowledge and system design are translated into practical application. During the internship at Encrypt Infotech LLP, implementation involved working on a live enterprise application, contributing to module development, resolving real-time client issues, and migrating legacy systems to modern frameworks.

The work carried out focused on improving system performance, ensuring data accuracy, and maintaining smooth operation of business processes through efficient coding practices and structured development methodologies.

6.1 Development Environment Setup

6.1.1 Hardware and System Requirements

The development work was carried out using a company-provided secured laptop configured with necessary tools and access permissions. The system was optimized for handling enterprise-level application development and testing.

6.1.2 Software Environment

The following software tools and frameworks were used:

- Java Development Kit (JDK) for backend programming
- Spring Boot Framework for application development
- Servlets and JSP for legacy system integration
- HTML, CSS for frontend design
- Jenkins for CI/CD automation
- Git for version control
- Cloud Platform for deployment

These tools enabled efficient development, debugging, and deployment of applications.

6.2 Module Implementation

The enterprise system consists of multiple interconnected modules. Each module was implemented and maintained with a focus on functionality, performance, and reliability.

6.2.1 Dashboard Module

The dashboard serves as the central interface of the application, displaying key business metrics such as delivered quantity, purchased amount, sales amount, and received amount. Implementation involved:

- Fetching real-time data from the database
- Displaying metrics using UI components
- Implementing date-based filtering functionality
- Ensuring accurate data visualization

The dashboard plays a vital role in decision-making by providing quick insights into system performance.

6.2.2 Supplier and Inventory Module

The Supplier and Inventory modules handle procurement and stock management operations. Implementation tasks included:

- Managing supplier records and purchase orders
- Tracking inventory levels and material movement
- Validating input data to prevent inconsistencies
- Ensuring synchronization between supplier and inventory data

These modules ensure efficient resource management and reduce operational errors.

6.2.3 Finance Module

The finance module manages transactions such as payments, receipts, and financial records. Implementation involved:

- Recording financial transactions
- Calculating totals and balances
- Maintaining transaction history
- Ensuring secure handling of financial data

This module is critical for maintaining financial transparency and accuracy.

6.2.4 CRM Module

The CRM module is responsible for managing customer information and interactions.

Implementation tasks included:

- Storing customer data
- Managing communication records
- Linking customer data with transactions
- Improving data retrieval efficiency

6.2.5 HR and Fleet Modules

The HR module manages employee data, while the fleet module handles vehicle-related operations.

Implementation included:

- Maintaining employee records
- Assigning roles and permissions
- Managing vehicle data and logistics
- Ensuring smooth coordination between modules

6.3 Feature Implementation and Enhancements

During the internship, several enhancements were implemented to improve system performance and usability.

6.3.1 Data Validation Mechanisms

Validation checks were implemented to ensure that user inputs were accurate and consistent. This prevented errors such as invalid entries and duplicate records.

6.3.2 Role-Based Access Control

Access control mechanisms were implemented to restrict user access based on roles. This ensured that sensitive data and functionalities were protected.

6.3.3 Performance Optimization

Backend logic was optimized to improve response time. Database queries were refined, and redundant operations were minimized.

6.4 Migration Implementation

6.4.1 Refactoring Legacy Code

Legacy applications built using Servlets and JSP were refactored into Spring Boot architecture. This involved restructuring code and adopting modern development practices.

6.4.2 Integration with Spring Boot

Spring Boot features such as dependency injection and auto-configuration were utilized to simplify development and improve maintainability.

6.4.3 Testing Migrated Modules

Each migrated module was thoroughly tested to ensure compatibility and correct functionality before deployment.

6.5 Client Issue Resolution Implementation

Real-time client issues were handled as part of the implementation phase.

6.5.1 Issue Identification

Client-reported issues were analyzed to identify root causes, including logic errors and data inconsistencies.

6.5.2 Debugging and Fixing

Code was debugged using appropriate tools, and necessary fixes were implemented to resolve issues.

6.5.3 Validation and Deployment

After fixing issues, modules were tested and deployed to ensure that the problem was resolved without affecting other functionalities.

6.6 Deployment Implementation

6.6.1 CI/CD Pipeline Execution

Jenkins pipelines were used to automate the build and deployment process. This reduced manual effort and ensured consistency.

6.6.2 Cloud Deployment Process

Applications were deployed on cloud platforms, ensuring high availability and scalability.

6.6.3 Monitoring and Maintenance

Post-deployment monitoring was carried out to track system performance and identify potential issues.

6.7 Challenges Faced During Implementation

Several challenges were encountered during the internship:

- Understanding complex legacy code
- Handling real-time production issues
- Ensuring data consistency during migration
- Managing multiple modules simultaneously

These challenges were overcome through continuous learning, mentorship, and systematic problem-solving.

6.8 Summary of Implementation

The implementation phase provided hands-on experience in developing and maintaining enterprise-level applications. The tasks performed contributed to improving system performance, enhancing user experience, and ensuring reliable operation of business processes.

The integration of modern frameworks, structured coding practices, and efficient deployment methods resulted in a robust and scalable application.

Chapter 7
EVALUATION AND RESULTS

The evaluation and results phase is a critical component of the internship report, as it reflects the effectiveness of the implemented solutions and the overall impact of the work carried out. During the internship at Encrypt Infotech LLP, evaluation was performed through systematic testing, performance analysis, and real-time observation of system behavior in a production environment.

The work involved handling live enterprise applications, where accuracy, reliability, and performance were of utmost importance. The evaluation process focused on validating system functionality, measuring performance improvements, ensuring data consistency, and assessing the effectiveness of migration from legacy systems to Spring Boot architecture.

7.1 Test Plan

The testing strategy adopted during the internship was designed to ensure that all modules functioned correctly and met the required business expectations. Since the application was actively used by clients, it was essential to validate every change thoroughly before deployment.

The test plan included defining test cases based on module functionality, expected inputs, and desired outputs. Each module, including Supplier PO, Dispatch, CRM, Inventory, and Finance, was tested independently as well as in integration with other modules. Special attention was given to edge cases, such as invalid inputs, missing data, and incorrect transactions, to ensure robustness of the system.

Testing was conducted in a structured manner, where each feature was verified against predefined acceptance criteria. This ensured that the system behaved as expected under various scenarios and minimized the risk of errors in production.

7.2 Testing Methodologies Applied

7.2.1 Unit Testing

Unit testing was performed at the code level to validate individual components such as service methods, controllers, and utility functions. Each function was tested independently to

ensure that it produced the correct output for given inputs. This helped in identifying logical errors early in the development process and ensured that the building blocks of the application were reliable.

7.2.2 Integration Testing

Integration testing was carried out to verify the interaction between different layers of the application, including controller, service, and database layers. This ensured that data flowed correctly across modules and that interdependent functionalities worked seamlessly. For example, transactions in the finance module were validated against inventory updates and customer records to ensure consistency.

7.2.3 Functional Testing

Functional testing focused on verifying whether the application met the specified business requirements. Each module was tested based on real-world use cases, such as generating invoices, tracking deliveries, updating inventory, and managing customer data. This ensured that the system was not only technically correct but also aligned with business needs.

7.2.4 Regression Testing

Regression testing was performed after implementing new features or bug fixes to ensure that existing functionalities were not affected. This was particularly important in an enterprise system where multiple modules are interconnected. By re-testing previously validated features, system stability was maintained throughout the development lifecycle.

7.2.5 Performance Testing

Performance testing was conducted to evaluate system responsiveness and efficiency, especially after migrating to Spring Boot. Metrics such as response time, system latency, and data processing speed were observed. The results indicated a noticeable improvement in performance due to optimized backend processing and reduced overhead in request handling.

7.3 Test Results and Observations

The testing process yielded positive results across all modules. Functional testing confirmed that all features operated as intended, while integration testing ensured smooth interaction

between modules. Regression testing showed that system updates did not introduce new issues, thereby maintaining stability.

Performance testing demonstrated improved response times, especially in data-intensive operations such as report generation and dashboard updates. The migration to Spring Boot played a significant role in enhancing system efficiency.

Overall, the test results validated the effectiveness of the implemented solutions and confirmed that the system met both technical and business requirements.

7.4 Performance Evaluation

The performance of the system was evaluated based on multiple parameters, including response time, scalability, and reliability. After migration from legacy servlet-based architecture to Spring Boot, the system exhibited significant improvements in handling concurrent requests and processing large volumes of data.

The optimized architecture reduced latency and improved user experience, particularly in modules that involved real-time data updates such as dashboard metrics and order tracking. The use of efficient database queries and structured coding practices contributed to faster execution and reduced system load.

These improvements ensured that the application could handle increased user activity without compromising performance.

7.5 Impact of Migration to Spring Boot

The migration from legacy systems to Spring Boot had a profound impact on the overall system performance and maintainability. The new architecture provided better modularity, allowing developers to manage and update individual components without affecting the entire system.

Spring Boot's auto-configuration and dependency injection features simplified development and reduced boilerplate code. This not only improved productivity but also enhanced code readability and maintainability.

From a performance perspective, the migration resulted in faster response times, improved scalability, and better resource utilization. These benefits made the system more robust and capable of supporting future enhancements.

7.6 Real-Time Client Feedback and System Behavior

One of the key aspects of evaluation during the internship was observing system behavior in a real-time production environment. Client feedback played an important role in assessing the effectiveness of implemented changes.

Clients reported improved system responsiveness and reduced downtime after updates. Issues that previously caused delays in operations were resolved efficiently, leading to increased client satisfaction.

The ability to handle real-time issues and deliver quick fixes demonstrated the reliability of the system and the effectiveness of the development and testing processes.

7.7 Data Accuracy and Consistency Analysis

Ensuring data accuracy and consistency was a critical requirement for the enterprise application. The evaluation process included verifying that all transactions, records, and reports were accurate and consistent across modules.

Validation mechanisms implemented during development helped prevent incorrect data entries and ensured that all records were synchronized. This was particularly important for financial transactions and inventory management, where inaccuracies could lead to significant business issues.

The system successfully maintained data integrity, which contributed to its reliability and trustworthiness.

7.8 Key Achievements and Outcomes

The internship resulted in several key achievements, both at the system level and personal level. From a system perspective, the successful migration to Spring Boot improved performance and maintainability. The implementation of validation mechanisms and role-based access control enhanced data security and reliability.

From an individual perspective, the internship provided hands-on experience in enterprise application development, real-time problem-solving, and client interaction. The ability to work on live systems and contribute to meaningful improvements was a significant learning outcome.

7.9 Limitations and Future Improvements

Despite the successful implementation and positive results, certain limitations were identified. Manual testing, although effective, can be time-consuming and prone to human error. The lack of automated testing frameworks limits the ability to quickly validate large-scale changes.

Future improvements could include the implementation of automated testing tools, integration of advanced monitoring systems, and further optimization of database operations. These enhancements would improve efficiency and reduce the risk of errors.

7.10 Summary of Evaluation and Results

The evaluation and results phase confirmed that the internship objectives were successfully achieved. The system demonstrated improved performance, reliability, and scalability after implementation and migration. Testing methodologies ensured that all modules functioned correctly, and real-time client feedback validated the effectiveness of the solutions.

Overall, the work carried out during the internship contributed significantly to the enhancement of the enterprise application and provided valuable insights into real-world software development practices.

Chapter 8

SOCIAL, LEGAL, ETHICAL, SUSTAINABILITY AND SAFETY ASPECTS

The development and deployment of enterprise-level software systems extend beyond technical implementation and require careful consideration of social, legal, ethical, sustainability, and safety aspects. During the internship at Encrypt Infotech LLP, these aspects played an important role in ensuring that the application not only functioned efficiently but also adhered to responsible and secure practices.

The enterprise application developed and maintained during the internship handles critical business data, including financial transactions, inventory records, and customer information. Therefore, maintaining integrity, confidentiality, and ethical use of data was of utmost importance.

8.1 Social Aspects

The enterprise application contributes significantly to improving business operations and organizational efficiency. By automating processes such as inventory management, financial tracking, and customer relationship management, the system reduces manual effort and minimizes human errors.

From a social perspective, the application enhances productivity and enables better decision-making through real-time data insights. Businesses can operate more efficiently, which in turn contributes to economic growth and improved service delivery.

Additionally, the system promotes digital transformation by encouraging organizations to adopt modern technologies. This reduces dependency on manual record-keeping and improves transparency in business operations.

However, the increased use of digital systems also requires users to adapt to new technologies. Proper training and user-friendly interfaces are essential to ensure smooth adoption and avoid resistance to change.

8.2 Legal Aspects

Legal compliance is a critical factor in the development and deployment of enterprise

applications. The system developed during the internship handles sensitive data such as financial records and customer information, making it essential to comply with data protection and privacy laws.

The application follows legal guidelines related to:

- Data privacy and protection
- Secure storage of user information
- Controlled access to sensitive data

Role-based access control ensures that only authorized users can access specific modules, thereby preventing unauthorized data usage. Additionally, proper logging mechanisms help track system activities, which is important for auditing and legal accountability.

Compliance with software licensing and intellectual property rights was also maintained. All tools, frameworks, and libraries used in development were properly licensed, ensuring adherence to legal standards.

8.3 Ethical Aspects

Ethical considerations are essential in software development, particularly when dealing with user data and business operations. During the internship, ethical practices were followed to ensure responsible use of technology.

Key ethical principles included:

- Maintaining confidentiality of client data
- Avoiding misuse of sensitive information
- Ensuring transparency in system operations
- Providing accurate and unbiased data processing

Developers were responsible for ensuring that the system did not manipulate or misrepresent data. Ethical coding practices were followed to avoid intentional or unintentional errors that could impact business decisions.

Respect for user privacy and data integrity was a primary focus, ensuring that the system operated in a fair and trustworthy manner.

8.4 Sustainability Aspects

Sustainability in software systems involves optimizing resource usage and minimizing environmental impact. The enterprise application contributes to sustainability by reducing the need for physical documentation and manual processes.

The use of digital records and automated workflows decreases paper consumption, supporting environmentally friendly practices. Additionally, cloud-based deployment reduces the need for physical infrastructure, leading to more efficient resource utilization. Efficient coding practices and optimized database queries help reduce computational overhead, which in turn lowers energy consumption. The migration to Spring Boot also improved system efficiency, contributing to long-term sustainability.
By enabling efficient resource management, the system supports sustainable business operations.

8.5 Safety Aspects

Safety in software systems primarily refers to data security and system reliability. The enterprise application implements several safety measures to ensure secure and stable operation.
These measures include:

- Authentication and authorization mechanisms
- Role-based access control
- Secure handling of financial and operational data
- Regular system monitoring

The system is designed to prevent unauthorized access and protect against potential security threats. Backup mechanisms and error-handling strategies ensure that data is not lost in case of system failures.
In addition, safe deployment practices using CI/CD pipelines help minimize risks during updates and ensure that only validated changes are introduced into the system.

8.6 Data Security and Privacy Considerations

Data security is one of the most critical aspects of enterprise applications. The system ensures that sensitive data such as financial records, customer details, and operational data are protected from unauthorized access.
Encryption techniques and secure communication protocols are used to safeguard data.
Access to critical data is restricted based on user roles, ensuring that only authorized personnel can view or modify sensitive information.
Regular monitoring and logging of system activities help in detecting potential security threats and maintaining system integrity.

8.7 Summary of Social, Legal, Ethical, Sustainability and Safety Aspects

The enterprise application developed and maintained during the internship adheres to important social, legal, ethical, sustainability, and safety considerations. These aspects ensure that the system is not only technically efficient but also responsible and secure in its operation.

By promoting digital transformation, ensuring legal compliance, maintaining ethical standards, supporting sustainability, and implementing strong safety measures, the application provides a comprehensive and reliable solution for business operations.

Chapter 9
CONCLUSION AND FUTURE SCOPE

The internship at Encrypt Infotech LLP provided a valuable opportunity to gain practical exposure to real-world software development and enterprise application management. The work carried out during the internship involved active participation in the development, maintenance, and enhancement of a live enterprise system used for managing business operations such as inventory, finance, customer relations, and production processes.

The primary objective of the internship was to understand the working of enterprise applications and contribute to their improvement through structured development and problem-solving approaches. This objective was successfully achieved through hands-on experience with technologies such as Java, Spring Boot, Servlets, and JSP, along with exposure to modern development practices including layered architecture, CI/CD pipelines, and cloud deployment.

Throughout the internship, significant contributions were made in areas such as module development, bug fixing, performance optimization, and migration of legacy systems to modern frameworks. The migration from servlet-based architecture to Spring Boot was a key highlight, as it improved system scalability, maintainability, and overall performance. The implementation of structured methodologies such as the V-Model ensured that all development activities were validated through proper testing, resulting in a reliable and stable system.

Working on a live enterprise application provided insights into the complexity of real-world systems, where multiple modules are interconnected and must function seamlessly. The experience of handling real-time client issues enhanced problem-solving skills and improved the ability to analyze and debug complex scenarios. The importance of writing efficient, maintainable, and secure code was clearly understood during this process.

The internship also emphasized the significance of teamwork, communication, and time management. Coordinating with team members, understanding client requirements, and delivering solutions within deadlines were essential aspects of the work environment. These experiences contributed to the development of professional skills required in the software industry.

From a technical perspective, the internship strengthened knowledge of backend

development, system architecture, and database interactions. It also provided an understanding of how enterprise systems are designed to handle large volumes of data while ensuring accuracy and consistency. The use of role-based access control, validation mechanisms, and secure coding practices highlighted the importance of data security and ethical considerations in software development.

The evaluation and results of the implemented solutions demonstrated improvements in system performance, response time, and reliability. The successful handling of client issues and positive feedback from users indicated the effectiveness of the development and testing processes. The system was able to operate efficiently in a real-time environment, meeting both technical and business requirements.

9.1 Learning Outcomes

The internship served as a bridge between academic learning and industry practices. It provided an opportunity to apply theoretical concepts in a practical environment and gain hands-on experience in enterprise application development.

Key learning outcomes include:

- Understanding of enterprise software architecture and design
- Hands-on experience with Java and Spring Boot
- Exposure to real-time debugging and issue resolution
- Knowledge of CI/CD pipelines and deployment processes
- Improved problem-solving and analytical skills
- Enhanced communication and teamwork abilities

These learnings have significantly contributed to both technical and professional development.

9.2 Future Scope

Although the system developed and maintained during the internship is robust and efficient, there are several areas where further improvements can be made.

One of the major areas of enhancement is the implementation of automated testing frameworks. Currently, manual testing is widely used, which can be time-consuming and prone to human error. Introducing automated testing tools would improve efficiency and ensure faster validation of system changes.

Another area of improvement is the integration of advanced monitoring and analytics tools.

These tools can provide deeper insights into system performance, user behavior, and potential bottlenecks, enabling proactive issue resolution.
The system can also be enhanced by adopting microservices architecture, which would further improve scalability and allow independent deployment of modules. This would make the application more flexible and easier to maintain.
In addition, incorporating modern frontend frameworks could improve user experience and make the interface more interactive and responsive. Enhancing security measures through advanced encryption techniques and multi-factor authentication would further strengthen data protection.

9.3 Final Remarks

In conclusion, the internship at Encrypt Infotech LLP was a highly enriching experience that provided practical exposure to the software development lifecycle in a professional environment. The opportunity to work on a live enterprise application, handle real-time challenges, and contribute to meaningful improvements has been invaluable.
The knowledge and skills gained during this internship have laid a strong foundation for a career in software development. The experience has not only enhanced technical expertise but also developed a deeper understanding of industry practices, making it a significant milestone in academic and professional growth.

REFERENCES

Books :

1. Walls, C., 2022, *Spring in Action*, Manning Publications
2. Pivotal Software, 2023, *Spring Boot Documentation*, Spring Official Website
3. Oracle Corporation, 2023, *Java Platform Documentation*, Oracle
4. Poutsma, A. et al., 2014, *Spring Framework Reference Documentation*, Spring
5. Fowler, M., 2002, *Patterns of Enterprise Application Architecture*, Addison-Wesley
6. Rajlich, V., & Bennett, K., 2000, *A Staged Model for the Software Life Cycle*, IEEE Computer
7. Sommerville, I., 2015, *Software Engineering*, Pearson
8. Mell, P., & Grance, T., 2011, *The NIST Definition of Cloud Computing*, NIST
9. Kim, G. et al., 2016, *The DevOps Handbook*, IT Revolution Press
10. Davenport, T. H., 1998, *Mission Critical: Realizing the Promise of Enterprise Systems*, Harvard Business School Press

Research Papers / Articles :

11. P. Mell and T. Grance, “The NIST Definition of Cloud Computing,” National Institute of Standards and Technology, 2011.
12. M. Fowler, “Microservices: A Definition of This New Architectural Term,” 2014.

Company Resources :

13. Internal documentation and training materials provided by Encrypt Infotech LLP.
14. Project source code, internal modules, and enterprise application dashboards accessed during the internship.

APPENDIX

The appendix section contains supplementary materials and supporting documents related to the internship work. These materials provide additional clarity and evidence of the tasks performed during the internship at Encrypt Infotech LLP.

A.1 Internship Completion Certificate

This section includes the official internship completion certificate issued by Encrypt Infotech LLP. The certificate serves as proof of successful completion of the internship and validates the duration, role, and contributions made during the internship period.

Encrypt Infotech

Certificate of Internship
TO WHOM IT MAY CONCERN.

This is to certify that **Ms. Rukmini Surve M.** has successfully completed the internship program as an **Associate Software Engineer** from 05.01.2026 to 12.04.2026. She worked on Product Development and Client Support during her internship.

She took keen interest in the work assigned and successfully completed it. During the period of internship, we found her to be punctual, hardworking and inquisitive.

We wish her luck and success in all her future endeavors.

Sincerely,

For Encrypt Infotech LLP.
SHYLAJA GOVINDARAJ
Managing Director

For ENCRYPT INFOTECH LLP
Shylaja
Authorised Signatory

ENCRYPT INFOTECH LLP
38/1, 38/2, 4th floor, Vinayaka high point, 1st main, 4th phase Yelahanka new town,
Bangalore- 560064 PH :9620720700,9916858790

A.3 Screenshots of Developed Application

This section includes screenshots of the enterprise application worked on during the internship. These screenshots provide visual evidence of the system interface, modules, and functionalities.

A.3.1 Dashboard Interface

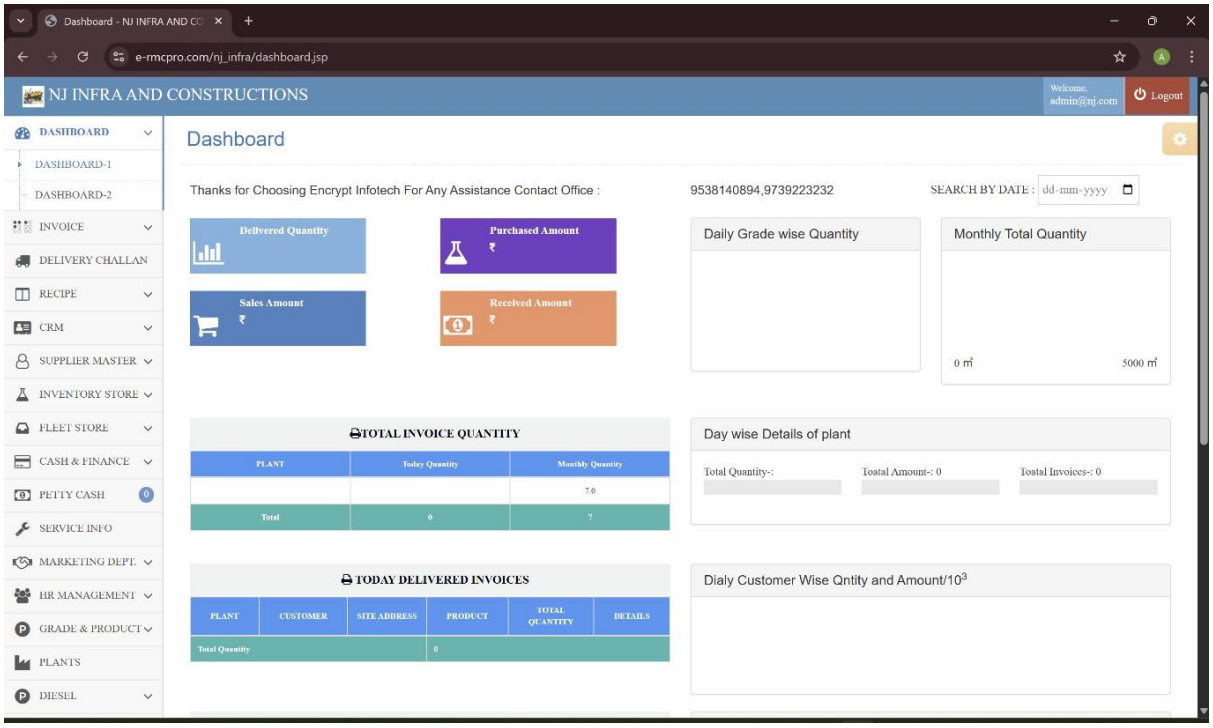

Figure 9.1: ERMC Dashboard 1

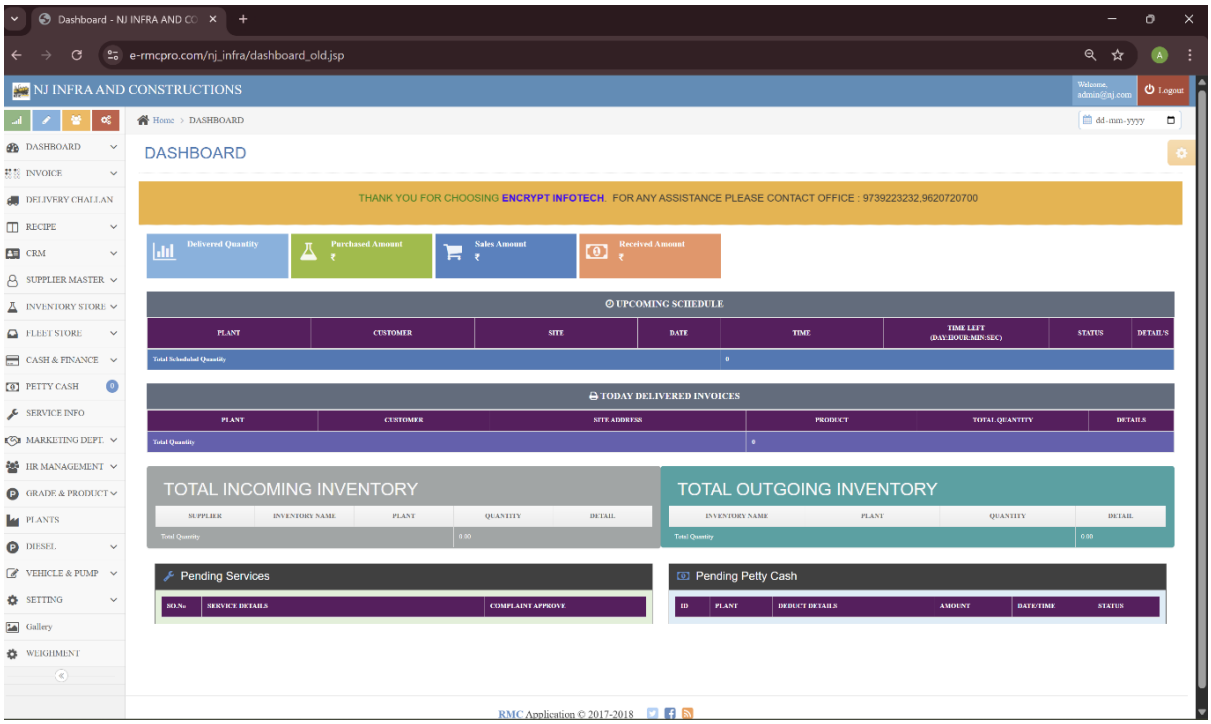

Figure 9.2: ERMC Dashboard 2

The dashboard displays key metrics such as delivered quantity, sales amount, purchased amount, and received amount. It acts as the central interface for monitoring system performance.

A.3.2 Module Navigation Interface

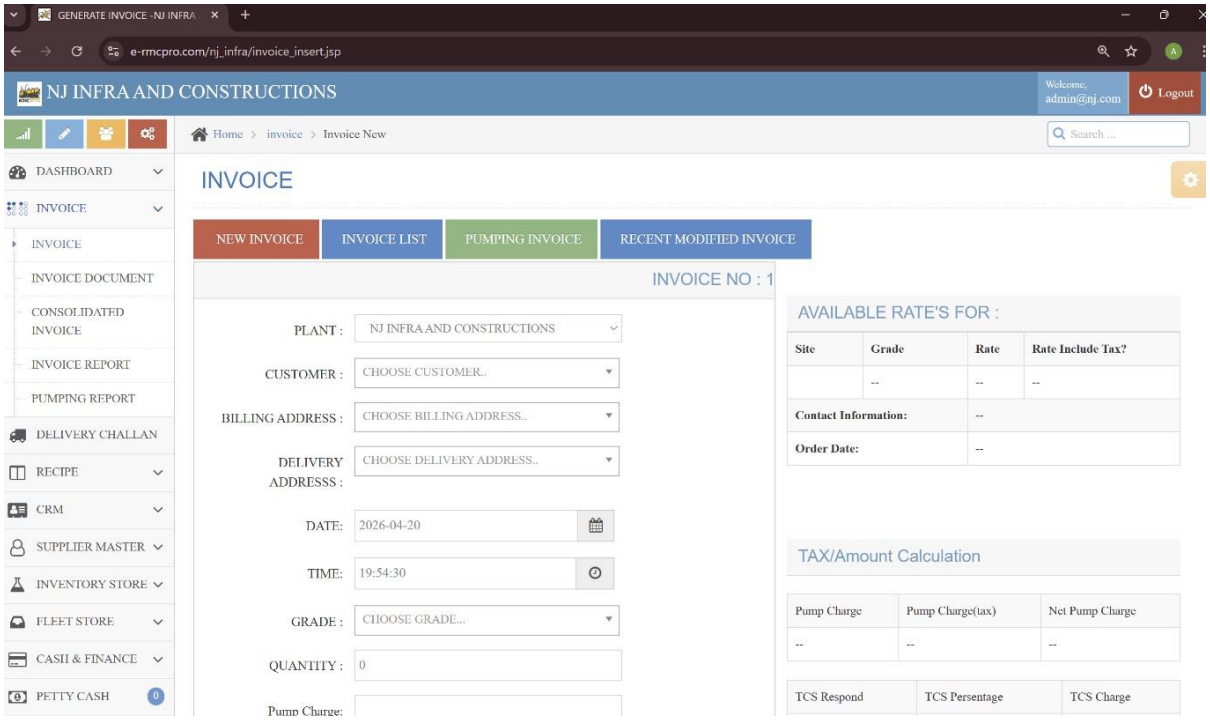

Figure 9.3: Navigation bar to access different modules.

The sidebar navigation provides access to different modules such as CRM, Inventory, Finance, HR, and Marketing. It ensures easy navigation across the system.